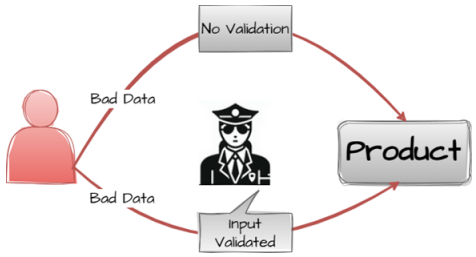
CWE-20: Improper Input Validation

Weakness ID: 20
Vulnerability Mapping: DISCOURAGED
Abstraction: Class

View customized information: Conceptual | Operational | Mapping Friendly | **Complete** | Custom

Description

The product receives input or data, but it does not validate or incorrectly validates that the input has the properties that are required to process the data safely and correctly.



Extended Description

Input validation is a frequently-used technique for checking potentially dangerous inputs in order to ensure that the inputs are safe for processing within the code, or when communicating with other components.

Input can consist of:

- raw data - strings, numbers, parameters, file contents, etc.
- metadata - information about the raw data, such as headers or size

Data can be simple or structured. Structured data can be composed of many nested layers, composed of combinations of metadata and raw data, with other simple or structured data.

Many properties of raw data or metadata may need to be validated upon entry into the code, such as:

- specified quantities such as size, length, frequency, price, rate, number of operations, time, etc.
- implied or derived quantities, such as the actual size of a file instead of a specified size
- indexes, offsets, or positions into more complex data structures
- symbolic keys or other elements into hash tables, associative arrays, etc.
- well-formedness, i.e. syntactic correctness - compliance with expected syntax
- lexical token correctness - compliance with rules for what is treated as a token
- specified or derived type - the actual type of the input (or what the input appears to be)
- consistency - between individual data elements, between raw data and metadata, between references, etc.
- conformance to domain-specific rules, e.g. business logic
- equivalence - ensuring that equivalent inputs are treated the same
- authenticity, ownership, or other attestations about the input, e.g. a cryptographic signature to prove the source of the data

Implied or derived properties of data must often be calculated or inferred by the code itself. Errors in deriving properties may be considered a contributing factor to improper input validation.

Common Consequences

Impact	Details
DoS: Crash, Exit, or Restart; DoS: Resource Consumption (CPU); DoS: Resource Consumption (Memory)	**Scope: Availability** An attacker could provide unexpected values and cause a program crash or arbitrary control of resource allocation, leading to excessive consumption of resources such as memory and CPU.

Read Memory; Read Files or Directories	**Scope: Confidentiality** An attacker could read confidential data if they are able to control resource references.
Modify Memory; Execute Unauthorized Code or Commands	**Scope: Integrity, Confidentiality, Availability** An attacker could use malicious input to modify data or possibly alter control flow in unexpected ways, including arbitrary command execution.

Potential Mitigations

Phase(s)	Mitigation
Architecture and Design	**Strategy: *Attack Surface Reduction*** Consider using language-theoretic security (LangSec) techniques that characterize inputs using a formal language and build "recognizers" for that language. This effectively requires parsing to be a distinct layer that effectively enforces a boundary between raw input and internal data representations, instead of allowing parser code to be scattered throughout the program, where it could be subject to errors or inconsistencies that create weaknesses. [REF-1109] [REF-1110] [REF-1111]
Architecture and Design	**Strategy: *Libraries or Frameworks*** Use an input validation framework such as Struts or the OWASP ESAPI Validation API. Note that using a framework does not automatically address all input validation problems; be mindful of weaknesses that could arise from misusing the framework itself (CWE-1173).
Architecture and Design; Implementation	**Strategy: *Attack Surface Reduction*** Understand all the potential areas where untrusted inputs can enter the product, including but not limited to: parameters or arguments, cookies, anything read from the network, environment variables, reverse DNS lookups, query results, request headers, URL components, e-mail, files, filenames, databases, and any external systems that provide data to the application. Remember that such inputs may be obtained indirectly through API calls.
Implementation	**Strategy: *Input Validation*** Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use a list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if the input is only expected to contain colors such as "red" or "blue." Do not rely exclusively on looking for malicious or malformed inputs. This is likely to miss at least one undesirable input, especially if the code's environment changes. This can give attackers enough room to bypass the intended validation. However, denylists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright. **Effectiveness: High**
Architecture and Design	For any security checks that are performed on the client side, ensure that these checks are duplicated on the server side, in order to avoid CWE-602. Attackers can bypass the client-side checks by modifying values after the checks have been performed, or by changing the client to remove the client-side checks entirely. Then, these modified values would be submitted to the server. Even though client-side checks provide minimal benefits with respect to server-side security, they are still useful. First, they can support intrusion detection. If the server receives input that should have been rejected by the client, then it may be an indication of an attack. Second, client-side error-checking can provide helpful feedback to the user about the expectations for valid input. Third, there may be a

	reduction in server-side processing time for accidental input errors, although this is typically a small savings.
Implementation	When your application combines data from multiple sources, perform the validation after the sources have been combined. The individual data elements may pass the validation step but violate the intended restrictions after they have been combined.
Implementation	Be especially careful to validate all input when invoking code that crosses language boundaries, such as from an interpreted language to native code. This could create an unexpected interaction between the language boundaries. Ensure that you are not violating any of the expectations of the language with which you are interfacing. For example, even though Java may not be susceptible to buffer overflows, providing a large argument in a call to native code might trigger an overflow.
Implementation	Directly convert your input type into the expected data type, such as using a conversion function that translates a string into a number. After converting to the expected data type, ensure that the input's values fall within the expected range of allowable values and that multi-field consistencies are maintained.
Implementation	Inputs should be decoded and canonicalized to the application's current internal representation before being validated (CWE-180, CWE-181). Make sure that your application does not inadvertently decode the same input twice (CWE-174). Such errors could be used to bypass allowlist schemes by introducing dangerous inputs after they have been checked. Use libraries such as the OWASP ESAPI Canonicalization control. Consider performing repeated canonicalization until your input does not change any more. This will avoid double-decoding and similar scenarios, but it might inadvertently modify inputs that are allowed to contain properly-encoded dangerous content.
Implementation	When exchanging data between components, ensure that both components are using the same character encoding. Ensure that the proper encoding is applied at each interface. Explicitly set the encoding you are using whenever the protocol allows you to do so.

Relationships

Relevant to the view "Research Concepts" (View-1000)

Nature	Type	ID	Name
ChildOf	P	707	Improper Neutralization
ParentOf	B	179	Incorrect Behavior Order: Early Validation
ParentOf	V	622	Improper Validation of Function Hook Arguments
ParentOf	B	1173	Improper Use of Validation Framework
ParentOf	B	1284	Improper Validation of Specified Quantity in Input
ParentOf	B	1285	Improper Validation of Specified Index, Position, or Offset in Input
ParentOf	B	1286	Improper Validation of Syntactic Correctness of Input
ParentOf	B	1287	Improper Validation of Specified Type of Input
ParentOf	B	1288	Improper Validation of Consistency within Input
ParentOf	B	1289	Improper Validation of Unsafe Equivalence in Input
PeerOf	C	345	Insufficient Verification of Data Authenticity
CanPrecede	B	22	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
CanPrecede	B	41	Improper Resolution of Path Equivalence
CanPrecede	C	74	Improper Neutralization of Special Elements in Output Used by a Downstream Component ('Injection')
CanPrecede	C	119	Improper Restriction of Operations within the Bounds of a Memory Buffer
CanPrecede	B	770	Allocation of Resources Without Limits or Throttling

Relevant to the view "Weaknesses for Simplified Mapping of Published Vulnerabilities" (View-

1003)

Nature	Type	ID	Name
MemberOf	V	1003	Weaknesses for Simplified Mapping of Published Vulnerabilities
ParentOf	V	129	Improper Validation of Array Index
ParentOf	B	1284	Improper Validation of Specified Quantity in Input

Relevant to the view "Architectural Concepts" (View-1008)

Nature	Type	ID	Name
MemberOf	C	1019	Validate Inputs

Relevant to the view "Seven Pernicious Kingdoms" (View-700)

Nature	Type	ID	Name
ParentOf	B	15	External Control of System or Configuration Setting
ParentOf	B	73	External Control of File Name or Path
ParentOf	V	102	Struts: Duplicate Validation Forms
ParentOf	V	103	Struts: Incomplete validate() Method Definition
ParentOf	V	104	Struts: Form Bean Does Not Extend Validation Class
ParentOf	V	105	Struts: Form Field Without Validator
ParentOf	V	106	Struts: Plug-in Framework not in Use
ParentOf	V	107	Struts: Unused Validation Form
ParentOf	V	108	Struts: Unvalidated Action Form
ParentOf	V	109	Struts: Validator Turned Off
ParentOf	V	110	Struts: Validator Without Form Field
ParentOf	V	111	Direct Use of Unsafe JNI
ParentOf	B	112	Missing XML Validation
ParentOf	V	113	Improper Neutralization of CRLF Sequences in HTTP Headers ('HTTP Request/Response Splitting')
ParentOf	C	114	Process Control
ParentOf	B	117	Improper Output Neutralization for Logs
ParentOf	C	119	Improper Restriction of Operations within the Bounds of a Memory Buffer
ParentOf	B	120	Buffer Copy without Checking Size of Input ('Classic Buffer Overflow')
ParentOf	B	134	Use of Externally-Controlled Format String
ParentOf	B	170	Improper Null Termination
ParentOf	B	190	Integer Overflow or Wraparound
ParentOf	B	466	Return of Pointer Value Outside of Expected Range
ParentOf	B	470	Use of Externally-Controlled Input to Select Classes or Code ('Unsafe Reflection')
ParentOf	V	785	Use of Path Manipulation Function without Maximum-sized Buffer

Modes Of Introduction

Phase	Note
Architecture and Design	
Implementation	REALIZATION: This weakness is caused during implementation of an architectural security tactic. If a programmer believes that an attacker cannot modify certain inputs, then the programmer might not perform any input validation at all. For example, in web applications, many programmers believe that cookies and hidden form fields can not be modified from a web browser (CWE-472), although they can be altered using a proxy or a custom program. In a client-server architecture, the programmer might assume that client-side security checks cannot be bypassed, even when a custom client could be written that skips those checks (CWE-602).

Applicable Platforms

Languages	
	Class: Not Language-Specific *(Often Prevalent)*

Likelihood Of Exploit

High

Demonstrative Examples

Example 1

This example demonstrates a shopping interaction in which the user is free to specify the quantity of items to be purchased and a total is calculated.

Example Language: **Java** *(bad code)*

```
...
public static final double price = 20.00;
int quantity = currentUser.getAttribute("quantity");
double total = price * quantity;
chargeUser(total);
...
```

The user has no control over the price variable, however the code does not prevent a negative value from being specified for quantity. If an attacker were to provide a negative value, then the user would have their account credited instead of debited.

Example 2

This example asks the user for a height and width of an m X n game board with a maximum dimension of 100 squares.

Example Language: **C** *(bad code)*

```
...
#define MAX_DIM 100
...
/* board dimensions */

int m,n, error;
board_square_t *board;
printf("Please specify the board height: \n");
error = scanf("%d", &m);
if ( EOF == error ){
  die("No integer passed: Die evil hacker!\n");
}
printf("Please specify the board width: \n");
error = scanf("%d", &n);
if ( EOF == error ){
  die("No integer passed: Die evil hacker!\n");
}
if ( m > MAX_DIM || n > MAX_DIM ) {
  die("Value too large: Die evil hacker!\n");
}
board = (board_square_t*) malloc( m * n * sizeof(board_square_t));
...
```

While this code checks to make sure the user cannot specify large, positive integers and consume too much memory, it does not check for negative values supplied by the user. As a result, an attacker can perform a resource consumption (CWE-400) attack against this program by specifying two, large negative values that will not overflow, resulting in a very large memory allocation (CWE-789) and possibly a system crash. Alternatively, an attacker can provide very large negative values which will cause an integer overflow (CWE-190) and unexpected behavior will follow depending on how the values are treated in the remainder of the program.

Example 3

The following example shows a PHP application in which the programmer attempts to display a user's birthday and homepage.

Example Language: **PHP** *(bad code)*

```
$birthday = $_GET['birthday'];
$homepage = $_GET['homepage'];
echo "Birthday: $birthday<br>Homepage: <a href=$homepage>click here</a>"
```

The programmer intended for $birthday to be in a date format and $homepage to be a valid URL. However, since the values are derived from an HTTP request, if an attacker can trick a victim into clicking a crafted URL with <script> tags providing the values for birthday and / or homepage, then the script will run on the client's browser when the web server echoes the content. Notice that even if the programmer were to defend the $birthday variable by restricting input to integers and dashes, it would still be possible for an attacker to provide a string of the form:

(attack code)

```
2009-01-09--
```

If this data were used in a SQL statement, it would treat the remainder of the statement as a comment. The comment could disable other security-related logic in the statement. In this case, encoding combined with input validation would be a more useful protection mechanism.

Furthermore, an XSS (CWE-79) attack or SQL injection (CWE-89) are just a few of the potential consequences when input validation is not used. Depending on the context of the code, CRLF Injection (CWE-93), Argument Injection (CWE-88), or Command Injection (CWE-77) may also be possible.

Example 4

The following example takes a user-supplied value to allocate an array of objects and then operates on the array.

Example Language: **Java** *(bad code)*

```
private void buildList ( int untrustedListSize ){
  if ( 0 > untrustedListSize ){
    die("Negative value supplied for list size, die evil hacker!");
  }
  Widget[] list = new Widget [ untrustedListSize ];
  list[0] = new Widget();
}
```

This example attempts to build a list from a user-specified value, and even checks to ensure a non-negative value is supplied. If, however, a 0 value is provided, the code will build an array of size 0 and then try to store a new Widget in the first location, causing an exception to be thrown.

Example 5

This Android application has registered to handle a URL when sent an intent:

Example Language: **Java** *(bad code)*

```
...
IntentFilter filter = new IntentFilter("com.example.URLHandler.openURL");
MyReceiver receiver = new MyReceiver();
registerReceiver(receiver, filter);
...

public class UrlHandlerReceiver extends BroadcastReceiver {
  @Override
  public void onReceive(Context context, Intent intent) {
    if("com.example.URLHandler.openURL".equals(intent.getAction())) {
      String URL = intent.getStringExtra("URLToOpen");
      int length = URL.length();

    ...
    }
  }
}
```

The application assumes the URL will always be included in the intent. When the URL is not present, the call to getStringExtra() will return null, thus causing a null pointer exception when length() is called.

Selected Observed Examples

Note: this is a curated list of examples for users to understand the variety of ways in which this weakness can be introduced. It is not a complete list of all CVEs that are related to this CWE entry.

Reference	Description
CVE-2024-37032	Large language model (LLM) management tool does not validate the format of a digest value (CWE-1287) from a private, untrusted model registry, enabling relative path traversal (CWE-23), a.k.a. Probllama
CVE-2022-45918	Chain: a learning management tool debugger uses external input to locate previous session logs (CWE-73) and does not properly validate the given path (CWE-20), allowing for filesystem path traversal using "../" sequences (CWE-24)
CVE-2021-30860	Chain: improper input validation (CWE-20) leads to integer overflow (CWE-190) in mobile OS, as exploited in the wild per CISA KEV.
CVE-2021-30663	Chain: improper input validation (CWE-20) leads to integer overflow (CWE-190) in mobile OS, as exploited in the wild per CISA KEV.
CVE-2021-22205	Chain: backslash followed by a newline can bypass a validation step (CWE-20), leading to eval injection (CWE-95), as exploited in the wild per CISA KEV.
CVE-2021-21220	Chain: insufficient input validation (CWE-20) in browser allows heap corruption (CWE-787), as exploited in the wild per CISA KEV.
CVE-2020-9054	Chain: improper input validation (CWE-20) in username parameter, leading to OS command injection (CWE-78), as exploited in the wild per CISA KEV.
CVE-2020-3452	Chain: security product has improper input validation (CWE-20) leading to directory traversal (CWE-22), as exploited in the wild per CISA KEV.
CVE-2020-3161	Improper input validation of HTTP requests in IP phone, as exploited in the wild per CISA KEV.
CVE-2020-3580	Chain: improper input validation (CWE-20) in firewall product leads to XSS (CWE-79), as exploited in the wild per CISA KEV.
CVE-2021-37147	Chain: caching proxy server has improper input validation (CWE-20) of headers, allowing HTTP response smuggling (CWE-444) using an "LF line ending"
CVE-2008-5305	Eval injection in Perl program using an ID that should only contain hyphens and numbers.
CVE-2008-2223	SQL injection through an ID that was supposed to be numeric.
CVE-2008-3477	lack of input validation in spreadsheet program leads to buffer overflows, integer overflows, array index errors, and memory corruption.
CVE-2008-3843	insufficient validation enables XSS
CVE-2008-3174	driver in security product allows code execution due to insufficient validation
CVE-2007-3409	infinite loop from DNS packet with a label that points to itself
CVE-2006-6870	infinite loop from DNS packet with a label that points to itself
CVE-2008-1303	missing parameter leads to crash
CVE-2007-5893	HTTP request with missing protocol version number leads to crash
CVE-2006-6658	request with missing parameters leads to information exposure
CVE-2008-4114	system crash with offset value that is inconsistent with packet size
CVE-2006-3790	size field that is inconsistent with packet size leads to buffer over-read

CVE-2008-2309	product uses a denylist to identify potentially dangerous content, allowing attacker to bypass a warning
CVE-2008-3494	security bypass via an extra header
CVE-2008-3571	empty packet triggers reboot
CVE-2006-5525	incomplete denylist allows SQL injection
CVE-2008-1284	NUL byte in theme name causes directory traversal impact to be worse
CVE-2008-0600	kernel does not validate an incoming pointer before dereferencing it
CVE-2008-1738	anti-virus product has insufficient input validation of hooked SSDT functions, allowing code execution
CVE-2008-1737	anti-virus product allows DoS via zero-length field
CVE-2008-3464	driver does not validate input from userland to the kernel
CVE-2008-2252	kernel does not validate parameters sent in from userland, allowing code execution
CVE-2008-2374	lack of validation of string length fields allows memory consumption or buffer over-read
CVE-2008-1440	lack of validation of length field leads to infinite loop
CVE-2008-1625	lack of validation of input to an IOCTL allows code execution
CVE-2008-3177	zero-length attachment causes crash
CVE-2007-2442	zero-length input causes free of uninitialized pointer
CVE-2008-5563	crash via a malformed frame structure
CVE-2008-5285	infinite loop from a long SMTP request
CVE-2008-3812	router crashes with a malformed packet
CVE-2008-3680	packet with invalid version number leads to NULL pointer dereference
CVE-2008-3660	crash via multiple "." characters in file extension

Weakness Ordinalities

Ordinality	Description
Primary	*(where the weakness exists independent of other weaknesses)*

Detection Methods

Method	Details
Automated Static Analysis	Some instances of improper input validation can be detected using automated static analysis. A static analysis tool might allow the user to specify which application-specific methods or functions perform input validation; the tool might also have built-in knowledge of validation frameworks such as Struts. The tool may then suppress or de-prioritize any associated warnings. This allows the analyst to focus on areas of the software in which input validation does not appear to be present. Except in the cases described in the previous paragraph, automated static analysis might not be able to recognize when proper input validation is being performed, leading to false positives - i.e., warnings that do not have any security consequences or require any code changes.
Manual Static Analysis	When custom input validation is required, such as when enforcing business rules, manual analysis is necessary to ensure that the validation is properly implemented.

Fuzzing	Fuzzing techniques can be useful for detecting input validation errors. When unexpected inputs are provided to the software, the software should not crash or otherwise become unstable, and it should generate application-controlled error messages. If exceptions or interpreter-generated error messages occur, this indicates that the input was not detected and handled within the application logic itself.
Automated Static Analysis - Binary or Bytecode	According to SOAR [REF-1479], the following detection techniques may be useful: Cost effective for partial coverage: • Bytecode Weakness Analysis - including disassembler + source code weakness analysis • Binary Weakness Analysis - including disassembler + source code weakness analysis **Effectiveness: SOAR Partial**
Manual Static Analysis - Binary or Bytecode	According to SOAR [REF-1479], the following detection techniques may be useful: Cost effective for partial coverage: • Binary / Bytecode disassembler - then use manual analysis for vulnerabilities & anomalies **Effectiveness: SOAR Partial**
Dynamic Analysis with Automated Results Interpretation	According to SOAR [REF-1479], the following detection techniques may be useful: Highly cost effective: • Web Application Scanner • Web Services Scanner • Database Scanners **Effectiveness: High**
Dynamic Analysis with Manual Results Interpretation	According to SOAR [REF-1479], the following detection techniques may be useful: Highly cost effective: • Fuzz Tester • Framework-based Fuzzer Cost effective for partial coverage: • Host Application Interface Scanner • Monitored Virtual Environment - run potentially malicious code in sandbox / wrapper / virtual machine, see if it does anything suspicious **Effectiveness: High**
Manual Static Analysis - Source Code	According to SOAR [REF-1479], the following detection techniques may be useful: Highly cost effective: • Focused Manual Spotcheck - Focused manual analysis of source • Manual Source Code Review (not inspections) **Effectiveness: High**
Automated Static Analysis - Source Code	According to SOAR [REF-1479], the following detection techniques may be useful: Highly cost effective: • Source code Weakness Analyzer

	• Context-configured Source Code Weakness Analyzer **Effectiveness: High**
Architecture or Design Review	According to SOAR [REF-1479], the following detection techniques may be useful: Highly cost effective: • Inspection (IEEE 1028 standard) (can apply to requirements, design, source code, etc.) • Formal Methods / Correct-By-Construction Cost effective for partial coverage: • Attack Modeling **Effectiveness: High**

Memberships

Nature	Type	ID	Name
MemberOf	V	635	Weaknesses Originally Used by NVD from 2008 to 2016
MemberOf	C	722	OWASP Top Ten 2004 Category A1 - Unvalidated Input
MemberOf	C	738	CERT C Secure Coding Standard (2008) Chapter 5 - Integers (INT)
MemberOf	C	742	CERT C Secure Coding Standard (2008) Chapter 9 - Memory Management (MEM)
MemberOf	C	746	CERT C Secure Coding Standard (2008) Chapter 13 - Error Handling (ERR)
MemberOf	C	747	CERT C Secure Coding Standard (2008) Chapter 14 - Miscellaneous (MSC)
MemberOf	C	751	2009 Top 25 - Insecure Interaction Between Components
MemberOf	C	872	CERT C++ Secure Coding Section 04 - Integers (INT)
MemberOf	C	876	CERT C++ Secure Coding Section 08 - Memory Management (MEM)
MemberOf	C	883	CERT C++ Secure Coding Section 49 - Miscellaneous (MSC)
MemberOf	C	994	SFP Secondary Cluster: Tainted Input to Variable
MemberOf	C	1005	7PK - Input Validation and Representation
MemberOf	C	1163	SEI CERT C Coding Standard - Guidelines 09. Input Output (FIO)
MemberOf	V	1200	Weaknesses in the 2019 CWE Top 25 Most Dangerous Software Errors
MemberOf	V	1337	Weaknesses in the 2021 CWE Top 25 Most Dangerous Software Weaknesses
MemberOf	C	1347	OWASP Top Ten 2021 Category A03:2021 - Injection
MemberOf	V	1350	Weaknesses in the 2020 CWE Top 25 Most Dangerous Software Weaknesses
MemberOf	C	1382	ICS Operations (& Maintenance): Emerging Energy Technologies
MemberOf	V	1387	Weaknesses in the 2022 CWE Top 25 Most Dangerous Software Weaknesses
MemberOf	C	1406	Comprehensive Categorization: Improper Input Validation
MemberOf	V	1425	Weaknesses in the 2023 CWE Top 25 Most Dangerous Software Weaknesses
MemberOf	V	1430	Weaknesses in the 2024 CWE Top 25 Most Dangerous Software Weaknesses
MemberOf	V	1435	Weaknesses in the 2025 CWE Top 25 Most Dangerous Software Weaknesses
MemberOf	C	1440	OWASP Top Ten 2025 Category A05:2025 - Injection

Vulnerability Mapping Notes

Usage	**DISCOURAGED** *(this CWE ID should not be used to map to real-world vulnerabilities)*
Reasons	Frequent Misuse, Frequent Misinterpretation, Abstraction
Rationale	CWE-20 is commonly misused in low-information vulnerability reports when lower-level CWEs could be used instead, or when more details about the vulnerability are available [REF-1287]. It is not useful for trend analysis. It is also a level-1 Class (i.e., a child of a Pillar). Finally, it is often used when the root cause issue is related to how input is incorrectly transformed, instead of "validated" to be correct as-is.
Comments	Within CWE, the "input validation" term focuses on the act of checking whether an input is already safe, which is different from other techniques that ensure safe processing of input. Carefully perform root-cause analysis to be sure that the issue is not due to techniques that attempt to transform potentially-dangerous input into something safe, such as filtering (CWE-790) - which attempts to remove dangerous inputs - or encoding/escaping (CWE-116), which attempts to ensure that the input is not misinterpreted when it is included in output to another component. If the issue is truly due to imroper input validation, consider using lower-level children such as Improper Use of Validation Framework (CWE-1173) or improper validation involving specific types or properties of input such as Specified Quantity (CWE-1284); Specified Index, Position, or Offset (CWE-1285); Syntactic Correctness (CWE-1286); Specified Type (CWE-1287); Consistency within Input (CWE-1288); or Unsafe Equivalence (CWE-1289).
Suggestions	**CWE-ID** / **Comment** CWE-1284 Specified Quantity CWE-1285 Specified Index, Position, or Offset CWE-1286 Syntactic Correctness CWE-1287 Specified Type CWE-1288 Consistency within Input CWE-1289 Unsafe Equivalence CWE-116 Improper Encoding or Escaping of Output CWE-790 Improper Filtering of Special Elements

Notes

Relationship

CWE-116 and CWE-20 have a close association because, depending on the nature of the structured message, proper input validation can indirectly prevent special characters from changing the meaning of a structured message. For example, by validating that a numeric ID field should only contain the 0-9 characters, the programmer effectively prevents injection attacks.

Multiple techniques exist to transform potentially dangerous input into something safe, which is different than "validation," which is a technique to check if an input is already safe. CWE users need to be cautious during root cause analysis to ensure that an issue is truly an input-validation problem.

Terminology

The "input validation" term is extremely common, but it is used in many different ways. In some cases its usage can obscure the real underlying weakness or otherwise hide chaining and composite relationships.

Some people use "input validation" as a general term that covers many different neutralization techniques for ensuring that input is appropriate, such as filtering, i.e., attempting to remove dangerous inputs (related to CWE-790); encoding/escaping, i.e., attempting to ensure that the input is not misinterpreted when it is included in output to another component (related to CWE-116); or canonicalization, which often indirectly removes otherwise-dangerous inputs. Others use the term in a narrower context to simply mean "checking if an input conforms to expectations without changing it." CWE uses this narrow interpretation.

Note that "input validation" has very different meanings to different people, or within different classification schemes. Caution must be used when referencing this CWE entry or mapping to it. For example, some weaknesses might involve inadvertently giving control to an attacker over an input

when they should not be able to provide an input at all, but sometimes this is referred to as input validation.

Finally, it is important to emphasize that the distinctions between input validation and output escaping are often blurred. Developers must be careful to understand the difference, including how input validation is not always sufficient to prevent vulnerabilities, especially when less stringent data types must be supported, such as free-form text. Consider a SQL injection scenario in which a person's last name is inserted into a query. The name "O'Reilly" would likely pass the validation step since it is a common last name in the English language. However, this valid name cannot be directly inserted into the database because it contains the "'" apostrophe character, which would need to be escaped or otherwise transformed. In this case, removing the apostrophe might reduce the risk of SQL injection, but it would produce incorrect behavior because the wrong name would be recorded.

Maintenance

As of 2020, this entry is used more often than preferred, and it is a source of frequent confusion. It is being actively modified for CWE 4.1 and subsequent versions.

Maintenance

Concepts such as validation, data transformation, and neutralization are being refined, so relationships between CWE-20 and other entries such as CWE-707 may change in future versions, along with an update to the Vulnerability Theory document.

Maintenance

Input validation - whether missing or incorrect - is such an essential and widespread part of secure development that it is implicit in many different weaknesses. Traditionally, problems such as buffer overflows and XSS have been classified as input validation problems by many security professionals. However, input validation is not necessarily the only protection mechanism available for avoiding such problems, and in some cases it is not even sufficient. The CWE team has begun capturing these subtleties in chains within the Research Concepts view (CWE-1000), but more work is needed.

Taxonomy Mappings

Mapped Taxonomy Name	Node ID	Fit	Mapped Node Name
7 Pernicious Kingdoms			Input validation and representation
OWASP Top Ten 2004	A1	CWE More Specific	Unvalidated Input
CERT C Secure Coding	ERR07-C		Prefer functions that support error checking over equivalent functions that don't
CERT C Secure Coding	FIO30-C	CWE More Abstract	Exclude user input from format strings
CERT C Secure Coding	MEM10-C		Define and use a pointer validation function
WASC	20		Improper Input Handling
Software Fault Patterns	SFP25		Tainted input to variable

Related Attack Patterns

CAPEC-ID	Attack Pattern Name
CAPEC-10	Buffer Overflow via Environment Variables
CAPEC-101	Server Side Include (SSI) Injection
CAPEC-104	Cross Zone Scripting
CAPEC-108	Command Line Execution through SQL Injection
CAPEC-109	Object Relational Mapping Injection
CAPEC-110	SQL Injection through SOAP Parameter Tampering
CAPEC-120	Double Encoding
CAPEC-13	Subverting Environment Variable Values
CAPEC-135	Format String Injection
CAPEC-136	LDAP Injection

CAPEC-14	Client-side Injection-induced Buffer Overflow
CAPEC-153	Input Data Manipulation
CAPEC-182	Flash Injection
CAPEC-209	XSS Using MIME Type Mismatch
CAPEC-22	Exploiting Trust in Client
CAPEC-23	File Content Injection
CAPEC-230	Serialized Data with Nested Payloads
CAPEC-231	Oversized Serialized Data Payloads
CAPEC-24	Filter Failure through Buffer Overflow
CAPEC-250	XML Injection
CAPEC-261	Fuzzing for garnering other adjacent user/sensitive data
CAPEC-267	Leverage Alternate Encoding
CAPEC-28	Fuzzing
CAPEC-3	Using Leading 'Ghost' Character Sequences to Bypass Input Filters
CAPEC-31	Accessing/Intercepting/Modifying HTTP Cookies
CAPEC-42	MIME Conversion
CAPEC-43	Exploiting Multiple Input Interpretation Layers
CAPEC-45	Buffer Overflow via Symbolic Links
CAPEC-46	Overflow Variables and Tags
CAPEC-47	Buffer Overflow via Parameter Expansion
CAPEC-473	Signature Spoof
CAPEC-52	Embedding NULL Bytes
CAPEC-53	Postfix, Null Terminate, and Backslash
CAPEC-588	DOM-Based XSS
CAPEC-63	Cross-Site Scripting (XSS)
CAPEC-64	Using Slashes and URL Encoding Combined to Bypass Validation Logic
CAPEC-664	Server Side Request Forgery
CAPEC-67	String Format Overflow in syslog()
CAPEC-7	Blind SQL Injection
CAPEC-71	Using Unicode Encoding to Bypass Validation Logic
CAPEC-72	URL Encoding
CAPEC-73	User-Controlled Filename
CAPEC-78	Using Escaped Slashes in Alternate Encoding
CAPEC-79	Using Slashes in Alternate Encoding
CAPEC-8	Buffer Overflow in an API Call
CAPEC-80	Using UTF-8 Encoding to Bypass Validation Logic

CAPEC-81	Web Server Logs Tampering
CAPEC-83	XPath Injection
CAPEC-85	AJAX Footprinting
CAPEC-88	OS Command Injection
CAPEC-9	Buffer Overflow in Local Command-Line Utilities

References

[REF-6]	Katrina Tsipenyuk, Brian Chess and Gary McGraw. *"Seven Pernicious Kingdoms: A Taxonomy of Software Security Errors".* NIST Workshop on Software Security Assurance Tools Techniques and Metrics. NIST. 2005-11-07. <https://samate.nist.gov/SSATTM_Content/papers/Seven%20Pernicious%20Kingdoms%20-%20Taxonomy%20of%20Sw%20Security%20Errors%20-%20Tsipenyuk%20-%20Chess%20-%20McGraw.pdf>.
[REF-166]	Jim Manico. *"Input Validation with ESAPI - Very Important".* 2008-08-15. <https://manicode.blogspot.com/2008/08/input-validation-with-esapi.html>. (*URL validated: 2023-04-07*)
[REF-45]	OWASP. *"OWASP Enterprise Security API (ESAPI) Project".* <https://owasp.org/www-project-enterprise-security-api/>. (*URL validated: 2025-07-24*)
[REF-168]	Joel Scambray, Mike Shema and Caleb Sima. *"Hacking Exposed Web Applications, Second Edition".* Input Validation Attacks. McGraw-Hill. 2006-06-05.
[REF-48]	Jeremiah Grossman. *"Input validation or output filtering, which is better?".* 2007-01-30. <https://blog.jeremiahgrossman.com/2007/01/input-validation-or-output-filtering.html>. (*URL validated: 2023-04-07*)
[REF-170]	Kevin Beaver. *"The importance of input validation".* 2006-09-06. <http://searchsoftwarequality.techtarget.com/tip/0,289483,sid92_gci1214373,00.html>.
[REF-7]	Michael Howard and David LeBlanc. *"Writing Secure Code".* Chapter 10, "All Input Is Evil!" Page 341. 2nd Edition. Microsoft Press. 2002-12-04. <https://www.microsoftpressstore.com/store/writing-secure-code-9780735617223>.
[REF-1109]	*"LANGSEC: Language-theoretic Security".* <http://langsec.org/>.
[REF-1110]	*"LangSec: Recognition, Validation, and Compositional Correctness for Real World Security".* <http://langsec.org/bof-handout.pdf>.
[REF-1111]	Sergey Bratus, Lars Hermerschmidt, Sven M. Hallberg, Michael E. Locasto, Falcon D. Momot, Meredith L. Patterson and Anna Shubina. *"Curing the Vulnerable Parser: Design Patterns for Secure Input Handling".* USENIX ;login:. 2017. <https://www.usenix.org/system/files/login/articles/login_spring17_08_bratus.pdf>.
[REF-1287]	MITRE. *"Supplemental Details - 2022 CWE Top 25".* Details of Problematic Mappings. 2022-06-28. <https://cwe.mitre.org/top25/archive/2022/2022_cwe_top25_supplemental.html#problematicMappingDetails>. (*URL validated: 2024-11-17*)
[REF-1479]	Gregory Larsen, E. Kenneth Hong Fong, David A. Wheeler and Rama S. Moorthy. *"State-of-the-Art Resources (SOAR) for Software Vulnerability Detection, Test, and Evaluation".* 2014-07. <https://www.ida.org/-/media/feature/publications/s/st/stateoftheart-resources-soar-for-software-vulnerability-detection-test-and-evaluation/p-5061.ashx>. (*URL validated: 2025-09-05*)

Content History

Submissions

Submission Date	Submitter	Organization
2006-07-19 *(CWE Draft 3, 2006-07-19)*	7 Pernicious Kingdoms	

Contributions

Contribution Date	Contributor	Organization

Submissions

2024-02-29 *(CWE 4.17, 2025-04-03)*	Abhi Balakrishnan *Contributed usability diagram concepts used by the CWE team.*	

Modifications

Previous Entry Names